

Week 2: Functions, Tests, and NumPy Review

Path Geometry and Verification · 2×75 -minute sessions

Opening engineering question. A point or robot appears to follow the displayed waypoints. What evidence is needed before we can say that the path calculation is correct?

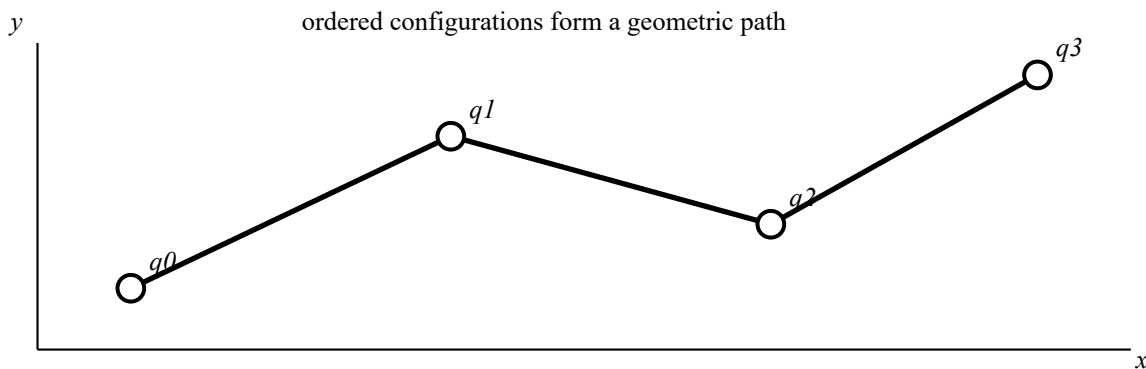


Figure 2.1. A geometric path is an ordered sequence of configurations connected by segments.

A motion-planning system eventually has to generate a path, check that it is safe, and execute it in a physical or virtual environment. Week 2 begins with the smaller programming problem on which those later tasks depend: representing a path, interpolating its segments, calculating its length, and verifying that the same meaning is preserved when one case becomes a NumPy batch. The path is supplied. Planning and collision checking arrive later in the course.

By the end of the chapter, the reader should be able to publish a numerical contract as a function, reject invalid input with a deliberate exception, translate that contract into tests, use array shapes and axes deliberately, and distinguish a computed path from the visualization that displays it.

2.1 One configuration begins as ordinary Python data

Before a path becomes an array or an object, it can be described with the Python constructs learned in the prerequisite course. A two-dimensional configuration has two coordinates. A tuple records one fixed pair, while a list collects an ordered number of configurations whose length may vary from path to path.

```
start = (0.0, 0.0)
corner = (3.0, 0.0)
goal = (3.0, 4.0)
waypoints = [start, corner, goal]

x_goal, y_goal = goal
first_two = waypoints[:2]
waypoints.append((4.0, 5.0))
```

The tuple `goal` is an immutable sequence: its two slots cannot be reassigned. The list `waypoints` is mutable: `append` adds another item. Unpacking binds `x_goal` and `y_goal` in the same order as the tuple. Slicing with `[:2]` includes indices 0 and 1 but excludes index 2.

Expression	Value	Programming meaning
<code>goal[0]</code>	<code>3.0</code>	select one coordinate by index
<code>goal</code>	<code>(3.0, 4.0)</code>	one fixed configuration
<code>waypoints[:2]</code>	first two tuples	select a path prefix
<code>len(waypoints)</code>	number of tuples	count configurations, not coordinates

The containers do not supply engineering meaning by themselves. The course contract states that each pair is ordered as (x, y) and that the list order is the path order. A program that silently exchanges `x` and `y` remains legal Python but no longer represents the declared path.

Iteration makes the representation computational

An ordered list becomes useful when one rule is applied consistently to consecutive items. The following loop uses `zip` to pair each configuration with the next one. The starred assignment is not needed here; explicit names make the data flow easier to inspect during review.

```
segments = []
for q_start, q_goal in zip(waypoints[:-1], waypoints[1:]):
    dx = q_goal[0] - q_start[0]
    dy = q_goal[1] - q_start[1]
    segments.append((dx, dy))

for index, (dx, dy) in enumerate(segments):
    print(index, dx, dy)
```

The two slices have equal length and are offset by one position. Pair 0 contains (`q0`, `q1`); pair 1 contains (`q1`, `q2`). The program therefore creates exactly $N - 1$ segments from N configurations. `enumerate` adds an index without changing the stored segment values.

Invariant. A valid path with N configurations has $N - 1$ consecutive segments. Preserving this relationship prevents missing or duplicated segments when the implementation is later vectorized.

A list comprehension could produce the same segment tuples in one expression. The explicit loop is shown first because the sequence of indexing, subtraction, and append operations is easier to trace. Concision is useful only after the meaning is stable.

```
segments_comp = [
    (q1[0] - q0[0], q1[1] - q0[1])
    for q0, q1 in zip(waypoints[:-1], waypoints[1:])
]
```

2.2 A reusable calculation needs a published contract

A formula inside a script can calculate one answer, but another module cannot safely reuse it unless the input, output, units, valid range, and failure behavior are stated. The function signature gives names and calling rules. The type hints and docstring document the intended values. Runtime code is still responsible for enforcing the contract.

```
def segment_length(
    start_xy: tuple[float, float],
    goal_xy: tuple[float, float],
) -> float:
    """Return Euclidean segment length in course coordinate units.

    Both inputs contain finite real values ordered as (x, y).
    Raise PathInputError when an input violates the public contract.
    """
    ...
```

Contract part	Published meaning
input	two configurations, each shaped as (2,) after normalization
output	one finite, nonnegative floating-point length
unit	course coordinate units
boundary	identical endpoints are valid and have length 0
failure	raise <code>PathInputError</code> ; do not clip, print, or return a sentinel
side effects	do not mutate input, print, plot, or write files

The type annotation does not prevent a caller from passing a string, a three-coordinate tuple, or an array containing NaN. It communicates intent to readers and tools. The function body must convert or reject actual runtime values according to the published boundary.

Validation separates representation errors from the calculation

Invalid path data can fail for several reasons. A value may not be numeric, a configuration may have the wrong shape, or a coordinate may be non-finite. These are representation errors. A domain-specific exception lets callers distinguish them from unrelated failures in NumPy or file handling.

```
class PathInputError(ValueError):
    """Raised when public path data violates the Week 2 contract."""

def _as_configuration(value, *, name: str) -> np.ndarray:
    raw = np.asarray(value, dtype=object)
    if raw.shape != (2,):
        raise PathInputError(f"{name} must have shape (2,)")
    number_types = (int, float, np.integer, np.floating)
    if any(
        isinstance(item, (bool, np.bool_)) or not isinstance(item, number_types)
        for item in raw.flat
    ):
        raise PathInputError(f"{name} must contain real coordinates")
    try:
        array = raw.astype(np.float64)
    except (TypeError, ValueError) as exc:
        raise PathInputError(f"{name} must contain real coordinates") from exc
    if not np.all(np.isfinite(array)):
        raise PathInputError(f"{name} must contain finite values")
    return array
```

The helper is internal because its leading underscore marks it as a module implementation detail rather than a student-facing API. The checks follow one order: convert the representation, verify the shape, verify finiteness, and only then compute. The public function reuses the helper so that every call applies the same boundary.

Do not silently repair input. Reshaping an arbitrary array, discarding a third coordinate, replacing NaN, or clipping a value would change the caller's data without an agreed rule. Reject the mismatch where it enters the package.

The scalar reference makes the numerical meaning inspectable

Once both configurations have been normalized, the Euclidean calculation is short enough to review line by line. The scalar function becomes the reference meaning against which later array implementations are checked.

$$d(q0, q1) = \text{sqrt}((x1 - x0)^2 + (y1 - y0)^2)$$

```
def segment_length(start_xy, goal_xy) -> float:
    start = _as_configuration(start_xy, name="start_xy")
    goal = _as_configuration(goal_xy, name="goal_xy")
    delta = goal - start
    return float(np.sqrt(np.sum(delta * delta)))
```

```
length = segment_length((0.0, 0.0), (3.0, 4.0))
print(length)
```

```
5.0
```

The result 5.0 follows from a 3-4-5 right triangle. This hand-checkable case is valuable because it tests the coordinate order and the squared-sum operation without depending on another implementation of the same function. The return conversion publishes a Python `float` rather than exposing a NumPy scalar as an accidental interface detail.

Identical endpoints form the boundary case. They are not an error: the displacement is zero and the segment length is zero. A path may contain a repeated configuration, although later planners may choose to remove it for efficiency.

```
assert segment_length((2.0, -1.0), (2.0, -1.0)) == 0.0
```

Pure computation stays separate from display and execution

The numerical module should answer a numerical question. Printing, plotting, saving a file, and commanding a simulator are different responsibilities. Separating them makes the calculation usable in tests, notebooks, command-line tools, later planners, and a virtual-physics adapter without changing its behavior.

Module	Responsibility	Student changes in Week 2
<code>path_geometry.py</code>	validate path data and return numerical results	yes
<code>demo.py</code>	call functions and print a small result	no
<code>visualize.py</code>	display the supplied path animation	no
tests	check the public contract and numerical relations	published tests: no; student tests: yes

```
# path_geometry.py: pure numerical result
points = interpolate_segment((0.0, 0.0), (3.0, 4.0), 5)

# visualize.py: supplied side effect
python -m ap_week02_path.visualize
# writes artifacts/path_geometry_preview.svg
```

The animation consumes the returned array. It does not prove how the array was produced and it does not replace numerical tests. Conversely, the numerical function does not need to know which simulator or plotting library will later display the path. Week 6 generalizes this separation into planner, validity-checker, and execution-adapter interfaces.

Course use of pure computation. A Week 2 numerical function returns values without printing, plotting, file I/O, global mutation, simulator calls, or mutation of caller-owned arrays.

2.3 Tests turn the contract into executable evidence

A test is not a second implementation of the entire package. It selects an input whose expected relation is known, calls the published function, and checks the result or failure behavior. The three basic categories exercise different parts of the contract.

Category	Example	Contract evidence
normal	(0,0) to (3,4)	ordinary distance is 5
boundary	identical endpoints	zero length is accepted
invalid	shape (3,) or a NaN coordinate	<code>PathInputError</code> is raised

```
def test_segment_length_3_4_5():
    actual = segment_length((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0)

def test_segment_length_accepts_zero_length():
    assert segment_length((1.5, -2.0), (1.5, -2.0)) == 0.0

def test_segment_length_rejects_nonfinite_coordinate():
    with pytest.raises(PathInputError, match="finite"):
        segment_length((0.0, np.nan), (1.0, 1.0))
```

The invalid test checks the public exception type and a stable fragment of its message. It does not depend on the helper name or the exact sequence of internal operations. Tests should protect published behavior while leaving room to refactor the implementation.

Read a failing test from the first mismatch

When pytest reports several failures, start with the first failure whose cause is understood. The report names the test, shows the assertion that failed, and prints the actual and expected values. The traceback then identifies the source line that produced the mismatch.

```
FAILED tests/test_published_contract.py::test_segment_length_3_4_5

E       assert 7.0 == 5.0 ± 5.0e-06
E       comparison failed
E       Obtained: 7.0
E       Expected: 5.0 ± 5.0e-06
```

An obtained value of 7 suggests that absolute coordinate differences were added rather than combined by the Euclidean norm. That is a hypothesis, not yet a conclusion. Inspect the cited source expression, repair the first observable mismatch, and rerun the focused test before rerunning the entire suite.

Floating-point equality needs an explicit policy

Many decimal results cannot be represented exactly in binary floating-point. Numerical tests therefore state a tolerance. `np.isclose` returns an elementwise Boolean result; `np.allclose` reduces an array comparison to one Boolean. A tolerance is not permission for an arbitrary mismatch. It should be small relative to the scale and numerical operations in the contract.

```
assert np.isclose(actual_length, expected_length, rtol=1e-12, atol=1e-12)
assert np.allclose(actual_points, expected_points, rtol=1e-12, atol=1e-12)
```

2.4 Arrays make the path shape explicit

A list of tuples is readable, but a numerical package benefits from one homogeneous array with a declared shape. Converting the waypoints creates one row per configuration and one column per coordinate.

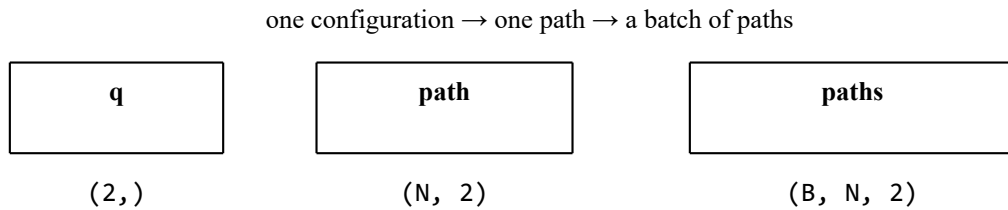


Figure 2.2. The array rank grows as one configuration becomes one path and then a batch.

```
path = np.asarray(
    [(0.0, 0.0), (3.0, 0.0), (3.0, 4.0)],
    dtype=np.float64,
)
```

```
print(path.ndim)
print(path.shape)
print(path[1, 0])
```

```
2
(3, 2)
3.0
```

The shape $(3, 2)$ means three configurations along axis 0 and two coordinates along axis 1. It does not mean three x values followed by two y values. Entry `path[i, j]` is coordinate `j` of configuration `i`. Writing the semantic meaning next to the shape is part of the data contract.

Indexing, slicing, views, and copies affect ownership

Basic NumPy slicing commonly returns a view that shares underlying data with the original array. This is efficient, but an assignment through the view may alter data owned by the caller. A public numerical function should not mutate its input unless mutation is explicitly part of the contract.

```
path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
middle_view = path[1:2]
middle_view[0, 0] = 99.0
print(path)
```

```
[[ 0.  0.]
 [99.  0.]
 [ 3.  4.]]
```

The slice keeps axis 0 because `1:2` selects a length-one range. It also shares memory, so the assignment changes `path`. An independent working array requires `copy()`. A calculation that only subtracts and reduces values often needs no copy at all; avoiding assignment is clearer and cheaper.

```
snapshot = path.copy()
_ = path_length(path)
assert np.array_equal(path, snapshot)
```

No-mutation contract. The equality test records the caller-owned input before the call and verifies that every entry is unchanged afterward.

2.5 Broadcasting expresses segment interpolation

Linear interpolation evaluates the point between two configurations at a parameter alpha. The formula includes the start at alpha = 0 and the goal at alpha = 1. A column of alpha values can be broadcast against a two-coordinate displacement. The public function accepts a Python or NumPy integer sample count $M \geq 2$ and rejects Boolean values; it must return exactly M rows.

$$q(\alpha) = (1 - \alpha) q_{start} + \alpha q_{goal}, \quad 0 \leq \alpha \leq 1$$

```
start = np.asarray((0.0, 0.0), dtype=np.float64) # (2,)
goal = np.asarray((3.0, 4.0), dtype=np.float64)  # (2,)
alpha = np.linspace(0.0, 1.0, 5)[: , None]       # (5, 1)
points = start[None, :] + alpha * (goal - start)[None, :]
print(points)
```

```
[[0.  0. ]
 [0.75 1. ]
 [1.5  2. ]
 [2.25 3. ]
 [3.  4. ]]
```

The trailing dimensions are compatible because 1 can expand to 2. Broadcasting produces shape (5, 2) without a Python loop. The endpoint rows should still be tested explicitly; a visually straight line does not reveal an off-by-one error that omitted the goal.

A scalar loop remains the reference for path length

The length of a polyline is the sum of the lengths of its consecutive segments. The formula does not replace the data contract: the implementation must still require a finite array of shape $(N, 2)$ with $N \geq 2$.

$$L(P) = \sum_{i=0}^{N-2} \|q_{i+1} - q_i\|_2$$

```
def path_length_scalar_reference(path_xy) -> float:
    path = np.asarray(path_xy, dtype=np.float64)
    total = 0.0
    for q0, q1 in zip(path[:-1], path[1:]):
        total += segment_length(q0, q1)
    return total

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
print(path_length_scalar_reference(path))
```

```
7.0
```

The first segment has length 3 and the second has length 4, so the path length is 7. The direct segment from start to goal would have length 5. Path length therefore depends on the ordered intermediate waypoints, not only on the endpoints. Later planners compare feasible paths using such geometric costs.

This scalar loop is deliberately retained as a reference even after a vectorized implementation exists. Agreement between structurally different implementations can expose an axis error that a single expected number does not.

Vectorization preserves the same consecutive-segment meaning

NumPy can create every segment displacement at once. The two slices keep the path order and both have shape $(N - 1, 2)$. Reduction along axis 1 combines x and y within each segment; the final sum combines segment lengths across the path.

```
def path_length(path_xy) -> float:
    path = _as_path(path_xy, name="path_xy")
    displacement = path[1:, :] - path[:-1, :]      # (N - 1, 2)
    squared = displacement * displacement          # (N - 1, 2)
    segment_lengths = np.sqrt(np.sum(squared, axis=1))
    return float(np.sum(segment_lengths))
```

Expression	Shape	Meaning
<code>path[1:] - path[:-1]</code>	$(N-1, 2)$	one displacement row per segment
<code>sum(..., axis=1)</code>	$(N-1,)$	squared length for each segment
<code>sqrt(...)</code>	$(N-1,)$	Euclidean segment lengths
final sum	scalar	total path length

Using axis 0 in the squared-sum step would combine different segments by coordinate and produce two values instead of $N - 1$ segment lengths. Shape tracing is therefore part of reasoning about correctness, not merely a debugging aid.

A batch adds one axis without changing one-path meaning

A fixed-size batch stores B paths, each with N configurations and two coordinates. The contract uses shape (B, N, 2). Axis 0 selects a path, axis 1 advances along its waypoints, and axis 2 selects x or y.

```
paths = np.array(
    [
        [[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]],
        [[0.0, 0.0], [0.0, 2.0], [2.0, 2.0]],
    ],
    dtype=np.float64,
)

displacement = paths[:, 1:, :] - paths[:, :-1, :] # (B, N-1, 2)
segment_lengths = np.sqrt(np.sum(displacement**2, axis=2))
lengths = np.sum(segment_lengths, axis=1)
print(lengths)
```

```
[7. 4.]
```

Axis	Index meaning	Reduction meaning
0	which path	combine or compare batch cases
1	which configuration/segment	sum segments into one path length
2	x or y coordinate	combine coordinates into one Euclidean segment length

The vectorized function must return shape (B,). It should not flatten the input, because flattening destroys the distinction among path, waypoint, and coordinate axes.

2.6 Properties verify more than one memorized answer

A few fixed examples are necessary but not sufficient. Geometric relations provide properties that should remain true across many inputs. Each property targets a different implementation risk.

Property	Expected relation	Likely defect exposed
endpoint	first and last interpolated rows equal the requested endpoints	off-by-one sampling
translation	adding one offset to every point preserves path length	absolute-position leakage
scalar-batch	batch entry <i>i</i> equals scalar result for path <i>i</i>	wrong axis or row alignment
permutation	permuting batch rows permutes outputs in the same way	hidden dependence on row number
no mutation	input arrays equal their snapshots after the call	view-based in-place assignment

```
offset = np.array([10.0, -4.0])
assert np.isclose(path_length(path + offset), path_length(path))

batch_actual = path_lengths(paths)
batch_expected = np.array([path_length(row) for row in paths])
assert np.allclose(batch_actual, batch_expected)

snapshot = paths.copy()
_ = path_lengths(paths)
assert np.array_equal(paths, snapshot)
```

These tests do not prove that every future input is correct, but they cover structural relations that many incorrect implementations violate. Hidden tests can vary the values while preserving the same published properties.

A complete worked example moves from data to evidence

The following small program uses all four published functions without a planner. It creates one segment, checks its interpolation, evaluates one path and then evaluates a fixed-size batch. Each printed value has an independent relation that can be tested.

```
start = (0.0, 0.0)
goal = (3.0, 4.0)
samples = interpolate_segment(start, goal, num_samples=5)

path_a = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
path_b = np.array([[0.0, 0.0], [0.0, 2.0], [2.0, 2.0]])
batch = np.stack([path_a, path_b], axis=0)

print(segment_length(start, goal))
print(samples.shape, samples[0], samples[-1])
print(path_length(path_a))
print(path_lengths(batch))

5.0
(5, 2) [0. 0.] [3. 4.]
7.0
[7. 4.]
```

The scalar distance is 5, but the path through the intermediate corner has length 7. The two values answer different questions. The interpolation output has five rows and includes both endpoints. The batch output has one value per input path. A visualization may display these arrays, but the numerical statements are already explicit and testable.

Terminology. A path specifies geometric configurations in order. It becomes a trajectory only after timing information is assigned. Week 2 computes paths, not trajectories.

2.7 Guided Lab A: publish and test the scalar behavior

Begin from the unmodified starter repository. Run the published suite before editing so that the baseline distinguishes missing student work from installation problems. Work from the repository root and record the first failure rather than copying only the final summary.

```
python -m pytest -q
python -m pytest tests/test_published_contract.py -k segment_length -x -q
python -m pytest tests/test_published_contract.py -k interpolation -x -q
```

Implement in dependency order. Inspect the provided `PathInputError`, add internal normalization helpers as needed, and then complete `segment_length`. Run the focused tests after each change. Only after scalar behavior is stable should interpolation be added.

Lab checkpoint	Evidence to retain
A1	baseline summary and first failure name
A2	normal, zero-length, wrong-shape, and non-finite scalar tests
A3	five-sample interpolation with exact endpoints
A4	clean <code>git diff</code> review before the first implementation commit

If a test fails, read the test name, assertion, obtained value, expected relation, and cited source line in that order. Do not broaden a tolerance or change a protected test to make a defect disappear.

Guided Lab B: preserve the scalar meaning in a NumPy batch

After scalar and interpolation tests pass, implement `path_length` and `path_lengths`. Write the shape transformations beside the vectorized expressions before executing them. Compare each batch result against a scalar reference and verify that the input arrays remain unchanged.

```
python -m pytest tests/test_published_contract.py -k path_length -x -q
```

```
python -m pytest -q
python -m ap_week02_path.demo
python -m ap_week02_path.visualize
python scripts/generate_artifacts.py
python scripts/check_submission.py
```

The supplied visualization writes an artifact derived from the computed path. Observe whether the marker begins at the first row, passes through the ordered waypoints, and finishes at the last row. Then return to the numerical evidence: shape, endpoint equality, path length, and no mutation. A smooth animation can still display an array produced by the wrong coordinate order.

Observation	Interpretation
animation completes	the provided display adapter consumed an array
endpoints match	interpolation includes the declared boundary rows
scalar and batch agree	the batch axes preserve one-path meaning
input snapshot is unchanged	the package respects caller ownership

The separate Assignment 2 document gives the required files, exact TODO order, submission evidence, and completion conditions. The final checker counts at least six top-level `test_...` functions in the student test file; parametrizing one function into several cases does not increase that function count. Complete all five prompts in the supplied engineering-note template. Do not begin by editing every TODO at once; complete and test one published behavior at a time.

2.8 Exercises and the transition to domain objects

The functions in this chapter deliberately use tuples and arrays because the week reviews procedural and numerical programming. As the planning problem grows, positional data becomes difficult to interpret. Week 3 introduces named objects that preserve the tested numerical functions while adding explicit state and responsibility.

Exercise 1

Predict the shape and value of `interpolate_segment((1, 2), (5, 6), 3)`. State which rows prove that the endpoints are included.

Exercise 2

Construct two different three-waypoint paths with the same endpoints but different lengths. Explain why a planner cannot rank them from the endpoints alone.

Exercise 3

A student computes `np.sum(displacement**2, axis=0)` for a path of shape `(5,2)`. Trace the resulting shape and explain why it does not represent four segment lengths.

Exercise 4

Design one property test that is different from the fixed 3-4-5 example. Identify the defect it is intended to expose.

Week 3 bridge. The next package will place validated configurations and paths inside named objects such as `Configuration`, `Path`, `PlanningProblem`, and `PlanResult`. The Week 2 functions remain the numerical foundation rather than being rewritten inside every class.